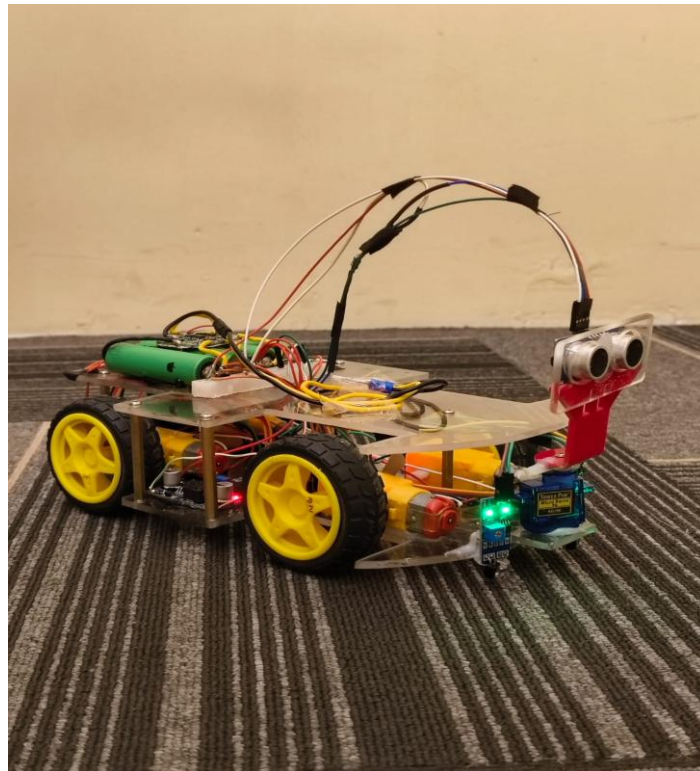


ENGINEERING PROJECT REPORT

The 5-in-1 Smart Multitask Robot Car

ESP32-S3 Single-Controller Architecture



Diploma in Electronics and Communication Engineering

Submitted by: Harsha Vardhan Reddy

September 2026

Table of Contents

Table of Contents.....	2
Chapter 1: Abstract and Executive Summary	4
1.1 Abstract.....	4
1.2 The Minimum-Controller Rule	4
1.3 Summary of Results	4
Chapter 2: Hardware Architecture and System Block Diagram.....	5
2.1 Component List	5
2.2 System Block Diagram.....	5
2.3 Functional Partitioning.....	6
Chapter 3: Pinout Matrix and Signal Conditioning	7
3.1 Complete Pinout Matrix.....	7
3.2 Servo Calibration.....	7
3.3 Echo-Line Voltage Divider	7
3.4 Line Sensor Logic.....	7
Chapter 4: Power System Engineering and Decoupling Analysis.....	9
4.1 Power Architecture	9
4.2 Grounding	9
4.3 Decoupling Capacitor Engineering History	9
4.4 Analysis	9
4.4.1 Failure 1: start-up inrush.....	9
4.4.2 Failure 2: oversized capacitor	9
4.4.3 Resolution: 330 uF / 25 V.....	9
Chapter 5: Kinematics and Floor Friction Tuning.....	11
5.1 Drive Configuration.....	11
5.2 The Turning Friction Challenge	11
5.3 Obstacle Threshold Scaling	11
5.4 The 360-Degree Drift Stunt.....	11
Chapter 6: Firmware Architecture and State Machine Design	12
6.1 Software Overview.....	12
6.2 State Machine	12
6.3 Non-Blocking Delays with yieldDelay()	12
6.4 Defensive Mode-Escape Checks	12
6.5 Dynamic Servo Detachment	12
6.6 Sensor Filtering	13
6.7 HTTP Route Map	13

Chapter 7: Human-Machine Interface and Speech Processing	14
7.1 Web Dashboard	14
7.2 Network Architecture	14
7.3 Voice Control with the Web Speech API.....	14
7.4 Phonetic Tolerance Engine	15
Chapter 8: Real-World Engineering Debugging Log.....	16
8.1 BMS Over-Current Cutoffs	16
8.2 Servo Jitter	16
8.3 Android 14 Split Routing	16
Chapter 9: Complete Production Source Code	17
Chapter 10: Conclusion and Performance Evaluation	28
10.1 Conclusion	28
10.2 Performance Evaluation.....	28
10.3 Observations on the Verified Firmware.....	28
10.4 Future Work	28

Chapter 1: Abstract and Executive Summary

1.1 Abstract

This report documents the design, construction, electrical debugging and firmware of the 5-in-1 Smart Multitask Robot Car, a four-wheel-drive, skid-steered robotic platform controlled entirely by a single ESP32-S3 microcontroller. The robot integrates five operating capabilities: manual joystick driving, voice-commanded driving, autonomous obstacle avoidance, infrared line following, and ultrasonic hand following, together with an automated 360-degree drift stunt. All capabilities, together with a locally hosted web dashboard and the safety logic that supervises them, execute on one dual-core Xtensa LX7 processor running at 240 MHz.

The project is a completed and physically verified build. Every pin assignment, component value and firmware constant reported here reflects the hardware as wired and the firmware as it runs.

1.2 The Minimum-Controller Rule

The governing design philosophy is the Minimum-Controller Rule: no function is delegated to a second processor if the primary controller can perform it. Heavier compute options were evaluated and deliberately eliminated during design. The final system therefore contains one microcontroller, and that controller alone performs motor control, sensor acquisition, Wi-Fi networking, HTTP serving, mode arbitration and watchdog-style safety exits.

The benefits of this approach are as follows:

- Lower electrical complexity: one logic supply rail, one ground reference and no inter-processor communication links to fail.
- Lower power demand from a 2S Li-ion pack, which matters because the pack protection circuit proved sensitive to current surges (Chapter 4).
- Simpler fault isolation: every observed fault could be traced to power, wiring or one firmware file.
- Deterministic behaviour through a single firmware state machine with instant abort paths (Chapter 6).

1.3 Summary of Results

Aspect	Outcome
Controller	One ESP32-S3, dual-core LX7 at 240 MHz; no co-processor
Operating modes	Manual, Obstacle Avoidance, Line Following, Hand Following (plus voice and drift stunt)
Control interface	HTML5 dashboard served from the robot on port 80; joystick, speed slider, tap-to-talk voice
Power	2S Li-ion (7.4 V nominal) with LM2596 buck to 5.0 V; 330 uF / 25 V decoupling capacitor
Key fixes	BMS over-current cutoffs, servo jitter, Android 14 split routing
Firmware structure	Non-blocking yieldDelay(), per-step mode-escape checks, dynamic servo detach

Chapter 2: Hardware Architecture and System Block Diagram

2.1 Component List

The build uses only the following components:

Component	Role
ESP32-S3 Development Module	Single system controller; Wi-Fi station, web server, all control logic
L298N dual H-bridge driver	Drives the left and right motor pairs (ENA/ENB for PWM speed, IN1-IN4 for direction)
4 x BO DC geared motors	Four-wheel drive, wired as a left pair and a right pair
SG90 micro servo	Pans the ultrasonic sensor head
HC-SR04 ultrasonic sensor	Distance measurement for obstacle avoidance and hand following
2 x TCRT5000 IR sensors	Left and right line detection (active-low)
2S Li-ion battery pack with BMS	Primary power, 7.4 V nominal, 8.4 V peak
LM2596 buck converter	Regulates the pack voltage down to 5.0 V logic rail
330 uF, 25 V capacitor	Decoupling across the L298N power terminals
1 kOhm and 2.2 kOhm resistors	Voltage divider on the HC-SR04 echo line

Motor Wiring Table

L298N Terminal	Channel	Connected Wires	Description
OUT1	Left Motor Channel	Front-Left Motor (+) & Rear-Left Motor (+)	Left side motor positive
OUT2	Left Motor Channel	Front-Left Motor (-) & Rear-Left Motor (-)	Left side motor negative
OUT3	Right Motor Channel	Front-Right Motor (+) & Rear-Right Motor (+)	Right side motor positive
OUT4	Right Motor Channel	Front-Right Motor (-) & Rear-Right Motor (-)	Right side motor negative
12V (VMS)	Power Channel	Battery (+) and capacitor (+)	Motor power input
GND	Common Ground	Battery (-), capacitor (-), ESP32-S3 GND	Common ground

2.2 System Block Diagram

Figure 2.1 shows the complete architecture. Power flows from the battery to the L298N motor supply and, through the LM2596, to the 5.0 V rail feeding the ESP32-S3, servo and sensors. The ESP32-S3 sends six control lines to the L298N, and the phone reaches the robot over the home Wi-Fi network.

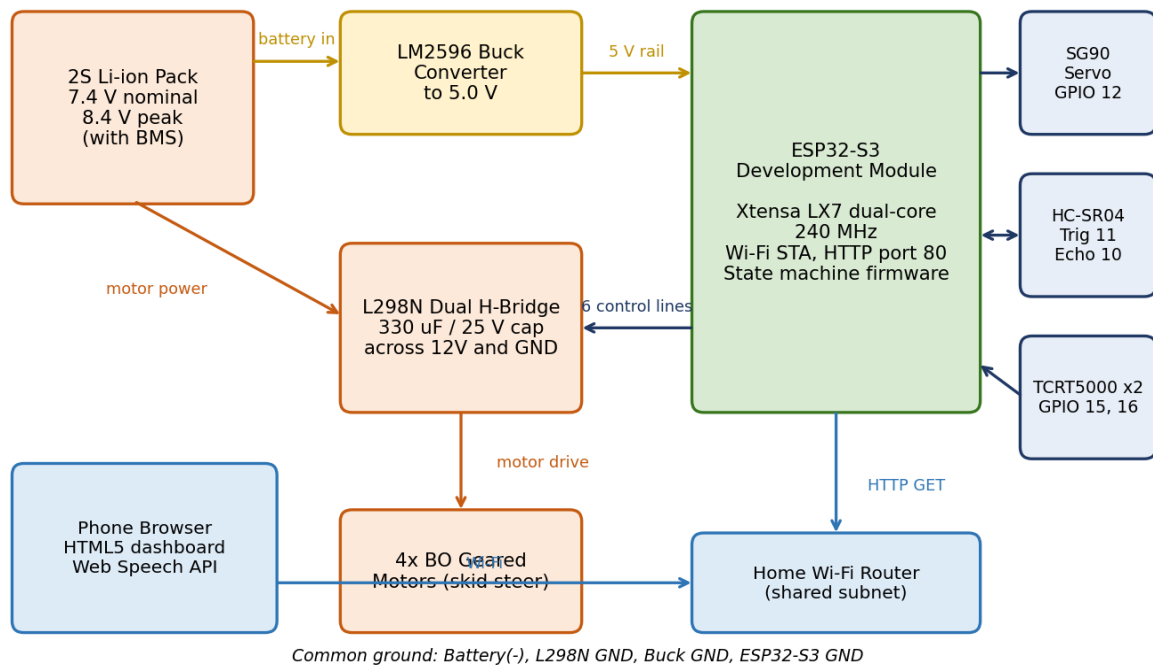


Figure 2.1: System block diagram of the 5-in-1 Smart Multitask Robot Car

2.3 Functional Partitioning

The system is divided into four cooperating subsystems: the power subsystem (battery, buck converter, decoupling), the actuation subsystem (L298N, four motors, servo), the sensing subsystem (ultrasonic ranger and two IR sensors) and the command subsystem (Wi-Fi, web server and browser dashboard). The ESP32-S3 is the only active controller in all four.

Chapter 3: Pinout Matrix and Signal Conditioning

3.1 Complete Pinout Matrix

The following eleven GPIO assignments are physically wired and verified. They match the constants declared in the firmware.

GPIO	Firmware name	Connected to	Function
45	pinENA	L298N ENA	Left motors speed (PWM)
48	pinIN1	L298N IN1	Left motors direction 1
47	pinIN2	L298N IN2	Left motors direction 2
21	pinIN3	L298N IN3	Right motors direction 1
14	pinIN4	L298N IN4	Right motors direction 2
13	pinENB	L298N ENB	Right motors speed (PWM)
12	pinServo	SG90 signal	Servo control
11	pinTrig	HC-SR04 Trig	Ultrasonic trigger output
10	pinEcho	HC-SR04 Echo (via divider)	Ultrasonic echo input
15	pinIR1	TCRT5000 left	Left line sensor (active-low)
16	pinIR2	TCRT5000 right	Right line sensor (active-low)

3.2 Servo Calibration

The SG90 is attached with a pulse range of 500 to 2400 microseconds. The physically calibrated straight-ahead position is exactly 135 degrees. The left sweep position is 180 degrees and the right sweep position is 90 degrees. These are stored as `ANGLE_CENTER`, `ANGLE_LEFT` and `ANGLE_RIGHT`.

3.3 Echo-Line Voltage Divider

The HC-SR04 is powered from the 5 V rail, so its echo output swings to 5 V, above the 3.3 V tolerance of the ESP32-S3 GPIO. The echo line is therefore conditioned by a resistive divider: a 1 kOhm series resistor from the sensor to GPIO 10, and a 2.2 kOhm resistor from GPIO 10 to ground.

The divider output is $V_{out} = 5.0 \text{ V} \times 2.2 / (1.0 + 2.2) = 3.44 \text{ V}$. This value is above the logic-high threshold of the ESP32-S3 and is the level used in the verified build. The series resistance also limits current into the pin during transitions.

3.4 Line Sensor Logic

Each TCRT5000 module is active-low, so a reading of 0 means the line is detected and 1 means it is not. The firmware decision table is:

Left IR	Right IR	Action
0	0	Move forward at AUTO_DRIVE_SPEED minus 10 (85 PWM)
1	0	Spin left at TURN_SPEED (220 PWM)
0	1	Spin right at TURN_SPEED (220 PWM)
1	1	Stop motors

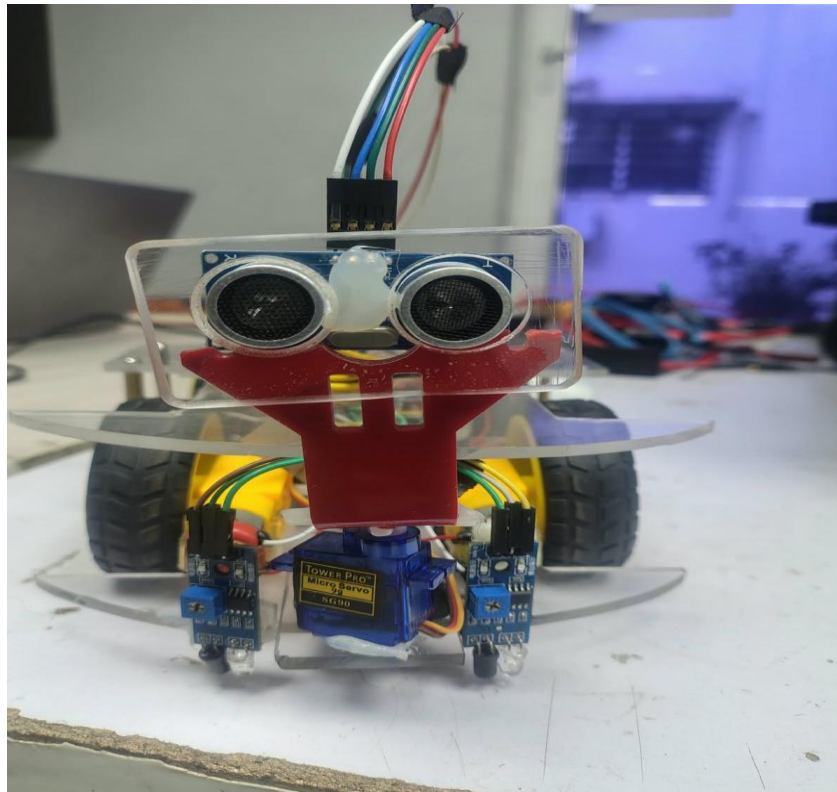


Fig. 1: Final Assembled Prototype of the Robot

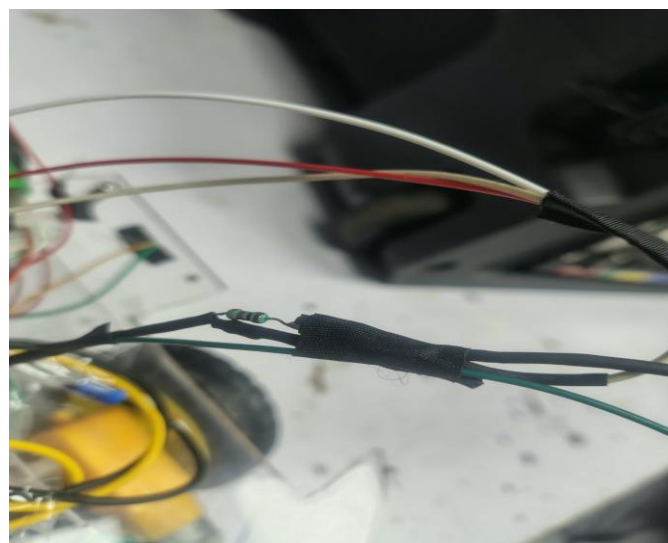


Fig. 2: Resistor Connection in the Robot Circuit

Chapter 4: Power System Engineering and Decoupling Analysis

4.1 Power Architecture

The primary source is a 2S Li-ion pack with a nominal voltage of 7.4 V and a peak of 8.4 V, protected by a battery management system (BMS). The pack feeds two loads directly: the L298N motor supply, and the LM2596 buck converter. The LM2596 steps the pack voltage down to a clean 5.0 V rail that supplies the ESP32-S3, the SG90 servo and the sensors.

4.2 Grounding

A single common ground is used. The following points are tied together: battery negative, L298N GND, LM2596 GND and ESP32-S3 GND. A shared reference is essential because the ESP32-S3 drives the L298N inputs directly and reads the sensors referenced to the same ground.

4.3 Decoupling Capacitor Engineering History

The selection of the decoupling capacitor was an iterative, physically tested process. Three configurations were evaluated.

Attempt	Configuration	Result
1	No suitable decoupling	Motor start-up inrush current tripped the BMS into over-current protection; output dropped to 0 V.
2	2200 uF capacitor	Ran for about 5 minutes, then tripped the BMS. Its very large charging inrush caused thermal accumulation in the BMS protection MOSFETs.
3	330 uF, 25 V capacitor	Stable. Suppresses inductive back-EMF spikes and motor electrical noise without overloading the BMS.

4.4 Analysis

4.4.1 Failure 1: start-up inrush

DC motors draw a current many times their running value at the instant of start, because the rotor is stationary and there is no back-EMF opposing the supply. With four motors starting together, the combined surge exceeded the BMS over-current threshold. The BMS opened its protection MOSFETs and the output fell to 0 V, which reset the whole robot.

4.4.2 Failure 2: oversized capacitor

A larger capacitor is not automatically better. A capacitor placed across the supply is discharged, so each connection to the pack draws a charging current limited only by the pack and BMS resistance. With 2200 uF this charging inrush was large. The observed failure mode, a trip after about five minutes rather than at once, is consistent with heat building up in the BMS protection MOSFETs until the protection threshold was reached.

4.4.3 Resolution: 330 uF / 25 V

A 330 uF, 25 V capacitor installed across the L298N power terminals (12 V and GND) provided enough local energy storage to absorb inductive back-EMF spikes from the motors and remove

electrical noise, while its charging inrush stayed low enough for the BMS. The 25 V rating gives ample margin over the 8.4 V peak pack voltage.

330µF 25V
Electrolytic Capacitor
8 x 12 mm | Lead Space 3 mm

High Reliability
Stable Performance
Wide Temperature Range
Long Lifespan

Key Specifications	
Capacitance	: 330 µF
Rated Voltage	: 25 V
Size (DxL)	: 8 x 12 mm
Lead Space	: 3 mm
Type	: Radial

Top View (vent)

Lead Space 3 mm

4.4.4 Use of 330µF Capacitor in the Project

The 330µF, 25V electrolytic capacitor is used in our project to stabilize the power supply and reduce sudden voltage fluctuations. It stores electrical energy and releases it when required, helping to provide a smoother and more stable voltage to the circuit. This improves the reliable operation of the Arduino, motor driver and other electronic components, especially when the motors start or change speed.

Chapter 5: Kinematics and Floor Friction Tuning

5.1 Drive Configuration

The robot is a four-wheel-drive skid-steer vehicle using four BO geared motors. The two left motors share one L298N channel and the two right motors share the other. Straight travel drives both sides in the same direction. Turning is achieved by driving the sides in opposite directions, so the robot pivots on the spot with no steered axle.

5.2 The Turning Friction Challenge

Driving straight requires relatively little torque because the tires roll. Turning a skid-steer vehicle is different: the tires must be dragged sideways across the floor, and this scrubbing friction is large. A speed setting that is adequate for cruising will stall or barely move the robot in a pivot.

The firmware resolves this with two speed regimes:

- **Cruising:** AUTO_DRIVE_SPEED is 95 PWM. In manual mode the speed slider adjusts activeDriveSpeed between 70 and 140 PWM, defaulting to 95.
- **Autonomous turning:** every autonomous pivot (obstacle avoidance, line following, hand following) uses TURN_SPEED, a constant 220 PWM.
- **Manual turning:** the /left and /right routes command the current slider speed plus a 25-point torque boost, capped at 220 PWM. At the default slider setting this is 120 PWM, and at the slider maximum of 140 it is 165 PWM.

Autonomous manoeuvres are timed rather than encoder-measured, so they depend on the robot actually completing the rotation. The high 220 PWM value provides the torque margin for that. The calibrated durations are TURN_DURATION of 850 ms for a standard turn (1.8 times that when both sides are blocked) and PIVOT_TIME of 800 ms for hand tracking.

5.3 Obstacle Threshold Scaling

The stopping distance for obstacle detection scales with speed. calculateDynamicThreshold() linearly interpolates between a 30 cm threshold at BASE_SPEED (70) and a 65 cm threshold at MAX_SPEED (120), and clamps outside that range. At the autonomous speed of 95 PWM the threshold is 47 cm.

5.4 The 360-Degree Drift Stunt

The drift stunt is an automated three-step sequence executed by executeDriftManeuver():

Step	Action	Parameters
1	Forward sprint	130 PWM for 750 ms (DRIFT_CHARGE_SPEED, DRIFT_CHARGE_DURATION)
2	Aggressive left spin	255 PWM (maximum) for 1000 ms (DRIFT_SPIN_SPEED, DRIFT_SPIN_DURATION)
3	Brake stop	stopMotors()

Chapter 6: Firmware Architecture and State Machine Design

6.1 Software Overview

The firmware is written in C++ for the Arduino-ESP32 framework using the WiFi, WebServer and ESP32Servo libraries. It has a conventional `setup()` and `loop()` structure. `loop()` services the web server and dispatches to one mode handler based on the current mode.

6.2 State Machine

A global enumeration `RobotMode` holds the current state, one of `MODE_MANUAL`, `MODE_OBSTACLE`, `MODE_LINE` and `MODE_GESTURE`. HTTP route handlers change the state, and `loop()` executes the matching routine. In manual mode the loop performs no motor action of its own, and motors respond only to HTTP commands.

Mode	Handler	Behaviour
<code>MODE_MANUAL</code>	(none in loop)	Motors driven directly by joystick, slider, voice and drift routes
<code>MODE_OBSTACLE</code>	<code>runObstacleAvoidance()</code>	Drive forward; on obstacle stop, optionally reverse, scan left and right, turn toward the more open side
<code>MODE_LINE</code>	<code>runLineFollowing()</code>	Two-sensor active-low line follower
<code>MODE_GESTURE</code>	<code>runHandFollowing()</code>	Keep a hand within 12 to 22 cm; follow up to 50 cm; sweep and pivot to reacquire

6.3 Non-Blocking Delays with `yieldDelay()`

The standard Arduino `delay()` blocks the processor, and while it runs the web server cannot service requests. In a robot whose autonomous routines contain many timed waits, this would make the dashboard unresponsive and would delay a stop command until the routine finished.

Every delay inside the autonomous routines and the drift stunt was therefore replaced by `yieldDelay()`. This function loops until the requested time has elapsed, calling `server.handleClient()` on each pass with a 1 ms delay. Incoming commands are handled continuously during waits, so the web server never locks up.

6.4 Defensive Mode-Escape Checks

Because `yieldDelay()` lets a new command arrive in the middle of a routine, each routine must notice that the mode has changed. Every autonomous loop therefore contains an exit check immediately after each delay:

```
if (currentMode != EXPECTED_MODE) { stopMotors(); return; }
```

Tapping any manual control or STOP on the phone changes the mode. At the next check the routine stops the motors and returns to `loop()`, aborting the autonomous action with no further steps executed.

6.5 Dynamic Servo Detachment

Motor switching generates electrical noise, including back-EMF, that coupled onto the servo signal and caused random 180-degree head swings. The firmware eliminates this by calling `headServo.detach()` whenever the servo is not needed, which stops the PWM signal entirely. Manual drive routes, the manual and line-following modes, and setup all detach the servo. Obstacle avoidance and hand following attach it on demand with `headServo.attach(pinServo, 500, 2400)`.

6.6 Sensor Filtering

`measureDistance()` triggers the HC-SR04 with a 10 microsecond pulse and reads the echo with a 15 ms timeout, returning -1 on timeout. Obstacle avoidance counts consecutive failed reads and stops after three, and otherwise substitutes the last good distance. Hand following counts absent-target readings and starts a left and right sweep after `ABSENCE_THRESHOLD` (5) misses.

6.7 HTTP Route Map

Route	Action
/	Serves the HTML dashboard
/forward, /backward	Manual drive at <code>activeDriveSpeed</code>
/left, /right	Manual spin at <code>activeDriveSpeed + 25</code> , capped at 220
/stop	Stop motors; manual mode
/set_speed?val=N	Update the manual drive speed
/drift	Run the drift manoeuvre
/mode_manual, /mode_obstacle, /mode_line, /mode_gesture	Switch operating mode and stop motors

Chapter 7: Human-Machine Interface and Speech Processing

7.1 Web Dashboard

The ESP32-S3 serves the dashboard directly from flash as a single HTML page. It has a virtual joystick, a speed slider from 70 to 140, mode buttons, a drift button, a tap-to-talk voice button and a telemetry panel showing the mode and voice status. The layout adapts to portrait and landscape orientation. The joystick uses a 35 percent dead zone and maps the dominant axis to forward, backward, left or right, sending a request only when the direction changes.

7.2 Network Architecture

The robot runs in Wi-Fi Station mode and joins the home router (SSID: Edge2), serving HTTP on port 80. The phone connects to the same router and opens the robot address printed on the serial monitor. The credentials are held in the `setup()` function of the firmware listing in Chapter 9.

7.3 Voice Control with the Web Speech API

Voice control runs entirely in the phone browser using the HTML5 Web Speech API (`webkitSpeechRecognition`). The recogniser is configured for non-continuous, final-result-only recognition in en-US. Tapping the microphone button starts listening, and tapping again stops it. When a transcript arrives it is lower-cased, shown on the telemetry panel and passed to a command parser, which issues the same fetch requests as the joystick.

This split keeps the microcontroller free of any audio load. The browser captures speech, the browser vendor cloud service transcribes it, and the robot only receives short HTTP commands. This is why the phone needs internet access during voice control (Section 8.3).

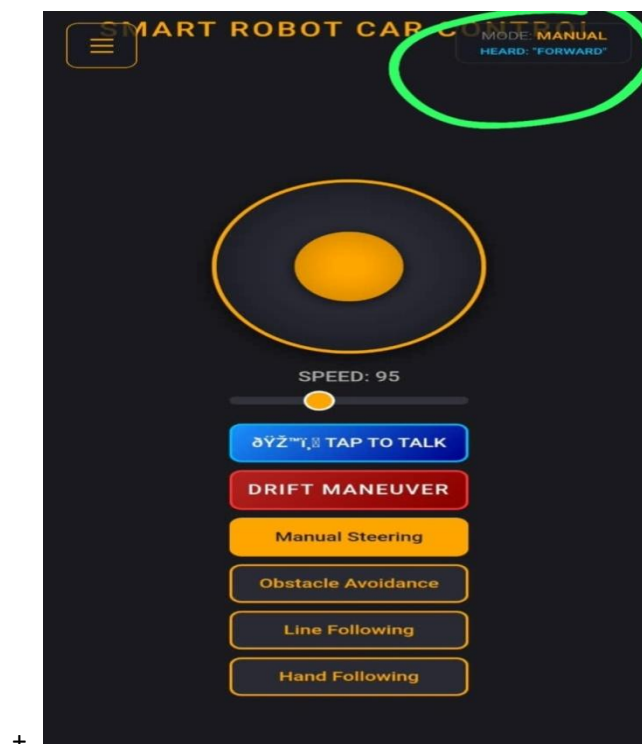


Fig.3: Smart Robot Car Mobile Control Interface

7.4 Phonetic Tolerance Engine

Speech engines vary with accent and background noise, and commonly return homophones or near-spellings. The parser matches substrings against a tolerant vocabulary:

Command	Accepted phrases
Forward	forward, forword, go straight, go
Backward	backward, backword, reverse, go back, back
Left	left, turn left
Right	right, write, rite, turn right
Stop	stop, brake, halt, top

Additional phrases select the drift stunt (drift, slide) and switch modes (obstacle or avoid, line or follow, gesture or hand, manual). Because matching uses substring inclusion, a short recognition error such as "top" for "stop" or "write" for "right" still produces the intended command. The conditions are evaluated in a fixed order, and the first match wins.

Chapter 8: Real-World Engineering Debugging Log

This chapter records the three major faults encountered on the physical robot and how each was resolved.

8.1 BMS Over-Current Cutoffs

Item	Detail
Symptom	Robot output dropped to 0 V and the system rebooted whenever the motors started.
Root cause	Motor start-up inrush current exceeded the battery BMS over-current threshold.
First remedy tried	A 2200 uF capacitor. It ran for about five minutes and then tripped the BMS again, because its large charging inrush caused thermal accumulation in the protection MOSFETs.
Resolution	A 330 uF, 25 V capacitor across the L298N power terminals (12 V and GND). Stable operation, and motor noise eliminated.
Lesson	Decoupling must be sized against the source, not only the load. A larger capacitor can be the cause of the fault.

8.2 Servo Jitter

Item	Detail
Symptom	The servo head swung randomly through 180 degrees while the robot drove.
Root cause	Back-EMF and switching noise from the motors corrupted the servo PWM signal line.
Resolution	Dynamic detachment: <code>headServo.detach()</code> in manual mode and line following, so the signal wire is switched off when the head is not required. The servo is attached only for obstacle avoidance and hand following.

8.3 Android 14 Split Routing

Item	Detail
Symptom	With the robot in standalone access-point mode and the phone mobile data enabled, motor commands did not reach the car.
Root cause	Android 14 split its routes. Local fetch requests were sent toward the cellular network, and lost, because the robot Wi-Fi had no internet access.
Resolution	Both the phone and the robot were connected to the home Wi-Fi network, placing them on one subnet. The phone keeps internet access for cloud speech transcription and sends local HTTP commands straight to the robot address.
Firmware consequence	The robot operates in Station mode (WIFI_STA) on port 80.

Chapter 9: Complete Production Source Code

The listing below is the verified master firmware, reproduced verbatim and unmodified. It runs on the ESP32-S3 in Arduino-ESP32 with the WiFi, WebServer and ESP32Servo libraries.

Note: this listing contains the Wi-Fi credentials of the project home network. Replace them before publishing or sharing this report.

```

/*
 6-in-1 Smart Robot Car - Stage 7: Mobile-Browser Voice Control (Home Wi-Fi Edition)
 Target: Customized ESP32-S3 (Station Mode)
*/

#include <WiFi.h>
#include <WebServer.h>
#include <ESP32Servo.h>

// --- Function Prototypes (Forward Declarations) ---
int calculateDynamicThreshold(int speed);
void moveForward(int speed);
void moveBackward(int speed);
void spinLeft(int speed);
void spinRight(int speed);
void stopMotors();
long measureDistance();
int scanDirection(int angle);
void executeDriftManeuver();
void runObstacleAvoidance();
void runLineFollowing();
void runHandFollowing();
void yieldDelay(unsigned long ms);

// --- Your Custom Pin Assignments ---
const int pinENA = 45; // Left motor speed
const int pinIN1 = 48; // Left motor dir 1
const int pinIN2 = 47; // Left motor dir 2
const int pinIN3 = 21; // Right motor dir 1
const int pinIN4 = 14; // Right motor dir 2
const int pinENB = 13; // Right motor speed

const int pinServo = 12; // Servo control pin
const int pinTrig = 11; // Ultrasonic Trigger
const int pinEcho = 10; // Ultrasonic Echo (via voltage divider)

const int pinIR1 = 15; // Left IR Sensor
const int pinIR2 = 16; // Right IR Sensor

// --- Central Mode Configuration ---
enum RobotMode {
  MODE_MANUAL,
  MODE_OBSTACLE,
  MODE_LINE,
  MODE_GESTURE
};
RobotMode currentMode = MODE_MANUAL;

// --- Speed & Calibration Parameters ---
const int AUTO_DRIVE_SPEED = 95; // Safe, calibrated speed for autonomous navigation
const int TURN_SPEED = 220; // High-torque turning speed
const int TURN_DURATION = 850; // Calibrated turn duration (ms)
const int PIVOT_TIME = 800; // Hand-tracking pivot duration (ms)

// Mutable manual speed driven by the slider (Default 95, ranges 70-140)
int activeDriveSpeed = 95;

const int BASE_SPEED = 70; // Adjusted Slow speed limit
const int MAX_SPEED = 120; // Adjusted Fast speed limit
const int BASE_THRESHOLD = 30;
const int MAX_THRESHOLD = 65;
int activeObstacleThreshold; // Calculated dynamically in setup

// --- Hand Tracking Parameters ---
const int DIST_TOO_CLOSE = 12;
const int DIST_START_FOLLOW = 22;
const int DIST_MAX_FOLLOW = 50;

// --- Servo Angles ---
const int ANGLE_CENTER = 135; // Calibrated center
const int ANGLE_LEFT = 180;
const int ANGLE_RIGHT = 90;
const int SERVO_DELAY = 400;

```

```

// Filtering variables
int consecutiveErrors = 0;
long lastGoodDistance = 100;
int absenceCounter = 0;
const int ABSENCE_THRESHOLD = 5;

// --- Stunt Configurations ---
const int DRIFT_CHARGE_SPEED = 130; // Speed for the initial 20cm run
const int DRIFT_SPIN_SPEED = 255; // Maximum torque for the 360-degree pivot
const int DRIFT_CHARGE_DURATION = 750; // Time (ms) to travel 20cm
const int DRIFT_SPIN_DURATION = 1000; // Time (ms) to execute the complete spin

// Web Server Port 80
WebServer server(80);

Servo headServo;

// --- HTML/CSS Web Dashboard with Joystick & Landscape Layout ---
const char* htmlDashboard = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-
scalable=no">
  <title>Smart Robot Car Dashboard</title>
  <style>
    /* Base page layout */
    body, html {
      font-family: Arial, sans-serif;
      text-align: center;
      background-color: #1a1a1f;
      color: white;
      margin: 0;
      padding: 0;
      overflow: hidden;
      height: 100vh;
    }

    /* Block text selection and callouts globally */
    body, html, .btn-mode, .btn, #joy-base, #joy-knob, .slider, .drawer-item {
      -webkit-touch-callout: none !important;
      -webkit-user-select: none !important;
      -khtml-user-select: none !important;
      -moz-user-select: none !important;
      -ms-user-select: none !important;
      user-select: none !important;
    }

    h1 { margin: 10px 0; font-size: 20px; color: #ffa500; text-transform: uppercase; letter-
spacing: 2px; }

    /* Master Layout Container */
    .dashboard-container { display: flex; flex-direction: column; align-items: center; justify-
content: center; height: calc(100vh - 40px); box-sizing: border-box; padding: 5px 15px; }

    /* Joystick Section */
    .joystick-panel { display: flex; justify-content: center; align-items: center; margin: 10px
0; }
    #joy-base { position: relative; width: 170px; height: 170px; background: radial-
gradient(circle, #2a2a35 40%, #15151b 100%); border: 3px solid #ffa500; border-radius: 50%; touch-
action: none; box-shadow: 0 4px 15px rgba(0,0,0,0.5); }
    #joy-knob { position: absolute; top: 50px; left: 50px; width: 70px; height: 70px;
background: radial-gradient(circle, #ffa500 20%, #cc8400 100%); border-radius: 50%; box-shadow: 0
2px 10px rgba(0,0,0,0.6); }

    /* Speed Control Slider */
    .slider-panel { width: 100%; max-width: 260px; margin: 5px 0 15px 0; }
    .slider-label { font-size: 14px; color: #aaa; margin-bottom: 5px; font-weight: bold; }
    .slider { -webkit-appearance: none; width: 100%; height: 8px; border-radius: 5px;
background: #33333d; outline: none; margin: 0; }
    .slider::-webkit-slider-thumb { -webkit-appearance: none; appearance: none; width: 20px;
height: 20px; border-radius: 50%; background: #ffa500; cursor: pointer; border: 2px solid #fff; }

    /* Mode Selector Panel */
    .mode-container { display: flex; flex-direction: column; gap: 8px; width: 100%; max-width:
260px; }
    .btn-mode { padding: 10px; background-color: #2b2b36; border: 2px solid #ffa500; border-
radius: 8px; color: #ffa500; font-size: 13px; font-weight: bold; cursor: pointer; transition: 0.1s;
width: 100%; box-sizing: border-box; }
    .btn-mode:active, .active-mode { background-color: #ffa500; color: #1a1a1f; }

    /* Action buttons */
    .btn-drift { background: linear-gradient(135deg, #b22222 0%, #8b0000 100%); border: 2px
solid #ff3333; color: white; font-size: 14px; letter-spacing: 1px; }
    .btn-drift:active { background: #ff0000; transform: scale(0.95); }

    /* Glowing Voice Microphone Button */
    .btn-voice { background: linear-gradient(135deg, #1e90ff 0%, #00008b 100%); border: 2px
solid #00bfff; color: white; }
    .btn-voice.listening { background: linear-gradient(135deg, #ff4500 0%, #b22222 100%);
border-color: #ff0000; box-shadow: 0 0 15px #ff0000; }

```

```

/* Floating HUD Telemetry */
.telemetry-panel { position: absolute; top: 15px; right: 15px; font-size: 11px; background:
rgba(26,26,36,0.8); border: 1px solid #333; padding: 6px 15px; border-radius: 8px; color: #888;
display: flex; flex-direction: column; align-items: flex-end; gap: 4px; }

/* Warning Banner */
.chrome-warn { display: none; font-size: 10px; color: #ff3333; background:
rgba(255,0,0,0.1); border: 1px dashed #ff3333; padding: 5px; border-radius: 5px; margin: 5px auto;
max-width: 280px; }

/* --- LANDSCAPE LAYOUT RESPONSIVENESS --- */
@media (orientation: landscape) {
  h1 { font-size: 18px; margin: 5px 0; }
  .dashboard-container { flex-direction: row; justify-content: space-around; width: 100%;
height: calc(100vh - 30px); padding: 5px 40px; }
  .joystick-panel { margin: 0; }
  #joy-base { width: 150px; height: 150px; }
  #joy-knob { width: 60px; height: 60px; top: 45px; left: 45px; }
  .right-column { display: flex; flex-direction: column; width: 50%; max-width: 320px;
gap: 10px; }
  .slider-panel { max-width: 100%; margin: 0; }
  .mode-container { width: 100%; max-width: 100%; gap: 6px; }
  .btn-mode { padding: 10px; font-size: 13px; }
  .chrome-warn { max-width: 100%; margin: 0; }
}
</style>
</head>
<body>
  <h1>Smart Robot Car</h1>

  <!-- Sidebar Toggle Menu -->
  <div id="menu-btn" onclick="toggleMenu(true)" style="position: absolute; top: 15px; left: 15px;
z-index: 800; width: 44px; height: 44px; background: rgba(26,26,36,0.8); border: 1px solid #ffa500;
border-radius: 8px; display: flex; align-items: center; justify-content: center; font-size: 20px;
color: #ffa500; cursor: pointer;">&#9776;</div>

  <!-- Top-Left Menu Drawer -->
  <div id="menu-drawer" style="display: none; flex-direction: column; position: fixed; top: 0;
left: 0; width: 220px; height: 100vh; background: rgba(11,11,14,0.95); border-right: 1px solid
#ffa500; z-index: 850; padding-top: 70px; text-align: left; box-sizing: border-box;">
    <div style="position: absolute; top: 15px; left: 15px; font-size: 22px; color: #b22222;
cursor: pointer;" onclick="toggleMenu(false)">&times;</div>
    <a href="#" class="drawer-item" onclick="triggerMode('/mode_manual', 'Manual
Steering')">MANUAL DRIVE</a>
    <a href="#" class="drawer-item" onclick="triggerMode('/mode_obstacle', 'Obstacle
Avoidance')">AUTO OBSTACLE</a>
    <a href="#" class="drawer-item" onclick="triggerMode('/mode_line', 'Line Following')">LINE
FOLLOW</a>
    <a href="#" class="drawer-item" onclick="triggerMode('/mode_gesture', 'Hand
Following')">HAND TRACK</a>
  </div>

  <!-- Floating HUD Telemetry -->
  <div class="telemetry-panel">
    <div>MODE: <b id="mode-telemetry" style="color: #ffa500;">MANUAL</b></div>
    <div id="vocal-hud" style="color: #00bfff; font-weight: bold; font-size: 9px;">VOCAL:
STANDBY</div>
  </div>

  <div class="dashboard-container">
    <!-- Left Side: Joystick -->
    <div class="joystick-panel">
      <div id="joy-base">
        <div id="joy-knob"></div>
      </div>
    </div>

    <!-- Right Side: Controls & Sliders -->
    <div class="right-column">
      <!-- Warning block for secure context issues -->
      <div id="chrome-warn-banner" class="chrome-warn">🔊 Mic access blocked. App permissions
denied OR check chrome://flags is enabled.</div>

      <!-- Speed Slider -->
      <div class="slider-panel">
        <div class="slider-label">SPEED: <span id="speed-val">95</span></div>
        <input type="range" min="70" max="140" value="95" class="slider" id="speed-slider"
oninput="updateSpeed(this.value)">
      </div>

      <!-- Mode Selector Panel -->
      <div class="mode-container">
        <button id="btn_voice" class="btn-mode btn-voice"
onclick="toggleVoiceRecognition()">🗣️ TAP TO TALK</button>
        <button id="btn_drift" class="btn-mode btn-drift" onclick="triggerDrift()">DRIFT
MANEUVER</button>
        <button id="m_manual" class="btn-mode active-mode"
onclick="changeMode('/mode_manual')">Manual Steering</button>
        <button id="m_obstacle" class="btn-mode"
onclick="changeMode('/mode_obstacle')">Obstacle Avoidance</button>
      </div>
    </div>
  </div>

```

```

        <button id="m_line" class="btn-mode" onclick="changeMode('/mode_line')">Line
Following</button>
        <button id="m_gesture" class="btn-mode" onclick="changeMode('/mode_gesture')">Hand
Following</button>
    </div>
</div>
</div>

<!-- Settings Overlay -->
<div id="settings-overlay">
    <div class="settings-card">
        <h3>Manual Controller Settings</h3>
        <div class="radio-row">
            <input type="radio" id="o-landscape" name="orientation" checked
onclick="setOrientation(true)">
            <label for="o-landscape">Force Landscape Layout</label>
        </div>
        <div class="radio-row">
            <input type="radio" id="o-portrait" name="orientation"
onclick="setOrientation(false)">
            <label for="o-portrait">Portrait Adaptive Layout</label>
        </div>
        <button class="btn-settings-close" onclick="closeSettings()">SAVE & APPLY</button>
    </div>
</div>

<script>
    const base = document.getElementById('joy-base');
    const knob = document.getElementById('joy-knob');
    const speedVal = document.getElementById('speed-val');
    const voiceBtn = document.getElementById('btn_voice');
    const vocalHud = document.getElementById('vocal-hud');
    let baseRect = base.getBoundingClientRect();

    let activeState = "STOP";

    window.addEventListener('resize', () => {
        baseRect = base.getBoundingClientRect();
    });

    // HTML5 Web Speech API Setup (Tap to Start / Tap to Stop)
    let SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;
    let recognition = null;
    let isListening = false;

    if (SpeechRecognition) {
        recognition = new SpeechRecognition();
        recognition.continuous = false;
        recognition.interimResults = false;
        recognition.lang = 'en-US';

        recognition.onstart = () => {
            isListening = true;
            voiceBtn.classList.add('listening');
            voiceBtn.innerText = "☐☐ LISTENING...";
            vocalHud.innerText = "VOCAL: LISTENING...";
        };

        recognition.onend = () => {
            isListening = false;
            voiceBtn.classList.remove('listening');
            voiceBtn.innerText = "☐☐ TAP TO TALK";
        };

        recognition.onerror = (e) => {
            console.error("Speech Error:", e);
            vocalHud.innerText = "VOCAL: ERROR";
        };

        recognition.onresult = (event) => {
            const command = event.results[0][0].transcript.toLowerCase();
            vocalHud.innerText = "HEARD: \"" + command.toUpperCase() + "\"";
            processVoiceCommand(command);
        };
    } else {
        voiceBtn.style.display = 'none';
    }

    function toggleVoiceRecognition() {
        if (!recognition) return;
        if (isListening) {
            recognition.stop();
        } else {
            recognition.start();
        }
    }

    function processVoiceCommand(phrase) {
        // Phonetic Tolerance Parser (Accepts alternative spellings like "forward" or
"backward")

```

```

    if (phrase.includes("forward") || phrase.includes("forword") || phrase.includes("go
straight") || phrase.includes("go")) {
        sendDirection("FORWARD");
    }
    else if (phrase.includes("backward") || phrase.includes("backword") ||
phrase.includes("reverse") || phrase.includes("go back") || phrase.includes("back")) {
        sendDirection("BACKWARD");
    }
    else if (phrase.includes("left") || phrase.includes("turn left")) {
        sendDirection("LEFT");
    }
    else if (phrase.includes("right") || phrase.includes("write") || phrase.includes("rite")
|| phrase.includes("turn right")) {
        sendDirection("RIGHT");
    }
    else if (phrase.includes("stop") || phrase.includes("brake") || phrase.includes("halt")
|| phrase.includes("top")) {
        sendDirection("STOP");
    }
    else if (phrase.includes("drift") || phrase.includes("slide")) {
        triggerDrift();
    }
    else if (phrase.includes("boost") || phrase.includes("speed")) {
        fetch('/boost');
    }
    else if (phrase.includes("obstacle") || phrase.includes("avoid")) {
        changeMode('/mode_obstacle');
    }
    else if (phrase.includes("line") || phrase.includes("follow")) {
        changeMode('/mode_line');
    }
    else if (phrase.includes("gesture") || phrase.includes("hand")) {
        changeMode('/mode_gesture');
    }
    else if (phrase.includes("manual")) {
        changeMode('/mode_manual');
    }
}

base.addEventListener('touchstart', handleStart, {passive: false});
base.addEventListener('touchmove', handleMove, {passive: false});
base.addEventListener('touchend', handleEnd, {passive: false});

function handleStart(e) {
    e.preventDefault();
    baseRect = base.getBoundingClientRect();
}

function handleMove(e) {
    e.preventDefault();
    const touch = e.touches[0];
    const centerX = baseRect.left + baseRect.width / 2;
    const centerY = baseRect.top + baseRect.height / 2;

    let x = touch.clientX - centerX;
    let y = touch.clientY - centerY;

    const maxDistance = baseRect.width / 2 - knob.clientWidth / 2;
    const distance = Math.sqrt(x*x + y*y);

    if (distance > maxDistance) {
        const angle = Math.atan2(y, x);
        x = Math.cos(angle) * maxDistance;
        y = Math.sin(angle) * maxDistance;
    }

    knob.style.transform = `translate(${x}px, ${y}px)`;
    evaluateDirection(x, y, maxDistance);
}

// Reset state and stop car on release
function handleEnd(e) {
    e.preventDefault();
    knob.style.transform = `translate(0px, 0px)`;
    sendDirection("STOP");
}

function evaluateDirection(x, y, maxDist) {
    const threshold = maxDist * 0.35;
    if (Math.abs(x) < threshold && Math.abs(y) < threshold) {
        sendDirection("STOP");
        return;
    }

    if (Math.abs(y) > Math.abs(x)) {
        if (y < 0) {
            sendDirection("FORWARD");
        } else {
            sendDirection("BACKWARD");
        }
    } else {
        if (x < 0) {

```

```

        sendDirection("LEFT");
    } else {
        sendDirection("RIGHT");
    }
}

function sendDirection(state) {
    if (activeState !== state) {
        activeState = state;
        let path = "/stop";
        if (state === "FORWARD") path = "/forward";
        if (state === "BACKWARD") path = "/backward";
        if (state === "LEFT") path = "/left";
        if (state === "RIGHT") path = "/right";

        if (state !== "STOP") {
            highlightMode('m_manual');
        }
        fetch(path);
    }
}

function updateSpeed(val) {
    speedVal.innerText = val;
    fetch(`/set_speed?val=${val}`);
}

// Action: Trigger drift sequence
function triggerDrift() {
    highlightMode('m_manual');
    fetch('/drift');
}

function toggleMenu(show) {
    document.getElementById('menu-drawer').style.display = show ? 'flex' : 'none';
}

// Configuration drawer controls
function openSettings() {
    toggleMenu(false);
    document.getElementById('settings-overlay').style.display = 'flex';
}

function closeSettings() {
    document.getElementById('settings-overlay').style.display = 'none';
}

function setOrientation(isLandscape) {
    let container = document.querySelector('.dashboard-container');
    if (isLandscape) {
        container.style.flexDirection = 'row';
    } else {
        container.style.flexDirection = 'column';
    }
}

function changeMode(path) {
    fetch(path);
    let modeId = 'm_manual';
    let label = "Manual";
    if (path === '/mode_obstacle') { modeId = 'm_obstacle'; label = "Obstacle"; }
    if (path === '/mode_line') { modeId = 'm_line'; label = "Line Follow"; }
    if (path === '/mode_gesture') { modeId = 'm_gesture'; label = "Hand Follow"; }
    highlightMode(modeId);
    document.getElementById('mode-telemetry').innerText = label;
}

function triggerMode(path, modeName) {
    fetch(path);
    toggleMenu(false);
    document.getElementById('mode-telemetry').innerText = modeName;

    let modeId = 'm_manual';
    if (path === '/mode_obstacle') modeId = 'm_obstacle';
    if (path === '/mode_line') modeId = 'm_line';
    if (path === '/mode_gesture') modeId = 'm_gesture';
    highlightMode(modeId);
}

function highlightMode(activeId) {
    document.querySelectorAll('.btn-mode').forEach(btn => {
        btn.classList.remove('active-mode');
    });
    document.getElementById(activeId).classList.add('active-mode');
}
</script>
</body>
</html>
)rawliteral";

```

```

// --- Dynamic Threshold Calculation ---
int calculateDynamicThreshold(int speed) {
  if (speed <= BASE_SPEED) return BASE_THRESHOLD;
  if (speed >= MAX_SPEED) return MAX_THRESHOLD;
  return BASE_THRESHOLD + ((speed - BASE_SPEED) * (MAX_THRESHOLD - BASE_THRESHOLD)) / (MAX_SPEED - BASE_SPEED);
}

// --- Non-Blocking Yield Delay Helper ---
void yieldDelay(unsigned long ms) {
  unsigned long start = millis();
  while (millis() - start < ms) {
    server.handleClient();
    delay(1);
  }
}

// --- Basic Motor Drives ---
void moveForward(int speed) {
  analogWrite(pinENA, speed);
  analogWrite(pinENB, speed);
  digitalWrite(pinIN1, HIGH);
  digitalWrite(pinIN2, LOW);
  digitalWrite(pinIN3, HIGH);
  digitalWrite(pinIN4, LOW);
}

void moveBackward(int speed) {
  analogWrite(pinENA, speed);
  analogWrite(pinENB, speed);
  digitalWrite(pinIN1, LOW);
  digitalWrite(pinIN2, HIGH);
  digitalWrite(pinIN3, LOW);
  digitalWrite(pinIN4, HIGH);
}

void spinLeft(int speed) {
  analogWrite(pinENA, speed);
  analogWrite(pinENB, speed);
  digitalWrite(pinIN1, LOW);
  digitalWrite(pinIN2, HIGH);
  digitalWrite(pinIN3, HIGH);
  digitalWrite(pinIN4, LOW);
}

void spinRight(int speed) {
  analogWrite(pinENA, speed);
  analogWrite(pinENB, speed);
  digitalWrite(pinIN1, HIGH);
  digitalWrite(pinIN2, LOW);
  digitalWrite(pinIN3, LOW);
  digitalWrite(pinIN4, HIGH);
}

void stopMotors() {
  analogWrite(pinENA, 0);
  analogWrite(pinENB, 0);
  digitalWrite(pinIN1, LOW);
  digitalWrite(pinIN2, LOW);
  digitalWrite(pinIN3, LOW);
  digitalWrite(pinIN4, LOW);
}

// --- Dynamic Distance Helper (Single Probe) ---
long measureDistance() {
  digitalWrite(pinTrig, LOW);
  delayMicroseconds(2);
  digitalWrite(pinTrig, HIGH);
  delayMicroseconds(10);
  digitalWrite(pinTrig, LOW);

  long duration = pulseIn(pinEcho, HIGH, 15000);
  if (duration == 0) return -1;
  return (duration * 0.0343) / 2;
}

// --- Dynamic Rotation Probe ---
int scanDirection(int angle) {
  if (headServo.attached() == false) headServo.attach(pinServo, 500, 2400);
  headServo.write(angle);
  yieldDelay(350);
  long distance = measureDistance();
  if (distance == -1) {
    yieldDelay(50);
    distance = measureDistance();
  }
  return distance;
}

// --- Automated Drift Maneuver ---
void executeDriftManeuver() {
  Serial.println("Executing 360 Drift...");
}

```

```

    moveForward(DRIFT_CHARGE_SPEED);
    yieldDelay(DRIFT_CHARGE_DURATION);
    spinLeft(DRIFT_SPIN_SPEED);
    yieldDelay(DRIFT_SPIN_DURATION);
    stopMotors();
}

// --- Autopilot: Obstacle Avoidance ---
void runObstacleAvoidance() {
    if (currentMode != MODE_OBSTACLE) return;
    if (headServo.attached() == false) headServo.attach(pinServo, 500, 2400);

    long currentDistance = measureDistance();
    if (currentMode != MODE_OBSTACLE) return;

    if (currentDistance == -1) {
        consecutiveErrors++;
        if (consecutiveErrors >= 3) {
            stopMotors();
            return;
        }
        currentDistance = lastGoodDistance;
    } else {
        consecutiveErrors = 0;
        lastGoodDistance = currentDistance;
    }

    if (currentDistance > activeObstacleThreshold) {
        moveForward(AUTO_DRIVE_SPEED);
        yieldDelay(30);
        if (currentMode != MODE_OBSTACLE) { stopMotors(); return; }
    }
    else {
        stopMotors();
        yieldDelay(400);
        if (currentMode != MODE_OBSTACLE) { stopMotors(); return; }

        if (currentDistance > 0 && currentDistance < 18) {
            moveBackward(BASE_SPEED);
            yieldDelay(750);
            if (currentMode != MODE_OBSTACLE) { stopMotors(); return; }
            stopMotors();
            yieldDelay(250);
            if (currentMode != MODE_OBSTACLE) { stopMotors(); return; }
        }

        int leftDistance = scanDirection(ANGLE_LEFT);
        if (currentMode != MODE_OBSTACLE) { stopMotors(); return; }
        int rightDistance = scanDirection(ANGLE_RIGHT);
        if (currentMode != MODE_OBSTACLE) { stopMotors(); return; }

        headServo.write(ANGLE_CENTER);
        yieldDelay(400);
        if (currentMode != MODE_OBSTACLE) { stopMotors(); return; }

        if (leftDistance == -1 && rightDistance == -1) {
            spinLeft(TURN_SPEED);
            yieldDelay(TURN_DURATION * 1.8);
        }
        else if (leftDistance >= rightDistance) {
            spinLeft(TURN_SPEED);
            yieldDelay(TURN_DURATION);
        }
        else {
            spinRight(TURN_SPEED);
            yieldDelay(TURN_DURATION);
        }
        if (currentMode != MODE_OBSTACLE) { stopMotors(); return; }
        stopMotors();
        yieldDelay(400);
    }
}

// --- Autopilot: Line Following ---
void runLineFollowing() {
    if (currentMode != MODE_LINE) return;
    if (headServo.attached()) headServo.detach();

    int leftIR = digitalRead(pinIR1);
    int rightIR = digitalRead(pinIR2);

    if (leftIR == 0 && rightIR == 0) {
        moveForward(AUTO_DRIVE_SPEED - 10);
    }
    else if (leftIR == 1 && rightIR == 0) {
        spinLeft(TURN_SPEED);
    }
    else if (leftIR == 0 && rightIR == 1) {
        spinRight(TURN_SPEED);
    }
    else if (leftIR == 1 && rightIR == 1) {
        stopMotors();
    }
}

```

```

    }
    yieldDelay(10);
    if (currentMode != MODE_LINE) { stopMotors(); return; }
}

// --- Autopilot: Hand Following ---
void runHandFollowing() {
    if (currentMode != MODE_GESTURE) return;
    if (headServo.attached() == false) headServo.attach(pinServo, 500, 2400);

    long distanceCenter = measureDistance();
    if (currentMode != MODE_GESTURE) return;

    if (distanceCenter > 0 && distanceCenter <= DIST_MAX_FOLLOW) {
        absenceCounter = 0;

        if (distanceCenter < DIST_TOO_CLOSE) {
            moveBackward(AUTO_DRIVE_SPEED);
        }
        else if (distanceCenter >= DIST_TOO_CLOSE && distanceCenter <= DIST_START_FOLLOW) {
            stopMotors();
        }
        else if (distanceCenter > DIST_START_FOLLOW && distanceCenter <= DIST_MAX_FOLLOW) {
            moveForward(AUTO_DRIVE_SPEED);
        }
        yieldDelay(50);
        if (currentMode != MODE_GESTURE) { stopMotors(); return; }
    }
    else {
        absenceCounter++;
        stopMotors();

        if (absenceCounter >= ABSENCE_THRESHOLD) {
            headServo.write(ANGLE_LEFT);
            yieldDelay(SERVO_DELAY);
            if (currentMode != MODE_GESTURE) { stopMotors(); return; }
            long distLeft = measureDistance();

            headServo.write(ANGLE_RIGHT);
            yieldDelay(SERVO_DELAY * 2);
            if (currentMode != MODE_GESTURE) { stopMotors(); return; }
            long distRight = measureDistance();

            headServo.write(ANGLE_CENTER);
            yieldDelay(SERVO_DELAY);
            if (currentMode != MODE_GESTURE) { stopMotors(); return; }

            absenceCounter = 0;

            if (distLeft > 0 && distLeft <= DIST_MAX_FOLLOW) {
                spinLeft(TURN_SPEED);
                yieldDelay(PIVOT_TIME);
                if (currentMode != MODE_GESTURE) { stopMotors(); return; }
                stopMotors();
                yieldDelay(300);
            }
            else if (distRight > 0 && distRight <= DIST_MAX_FOLLOW) {
                spinRight(TURN_SPEED);
                yieldDelay(PIVOT_TIME);
                if (currentMode != MODE_GESTURE) { stopMotors(); return; }
                stopMotors();
                yieldDelay(300);
            }
        }
        else {
            yieldDelay(50);
            if (currentMode != MODE_GESTURE) { stopMotors(); return; }
        }
    }
}

// --- Main Setup Loop ---
void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("--- Starting Stage 7 Mobile Voice Setup ---");

    activeObstacleThreshold = calculateDynamicThreshold(AUTO_DRIVE_SPEED);

    // Initialize motor outputs
    pinMode(pinENA, OUTPUT);
    pinMode(pinIN1, OUTPUT);
    pinMode(pinIN2, OUTPUT);
    pinMode(pinIN3, OUTPUT);
    pinMode(pinIN4, OUTPUT);
    pinMode(pinENB, OUTPUT);

    // Sensors
    pinMode(pinIR1, INPUT);
    pinMode(pinIR2, INPUT);
    pinMode(pinTrig, OUTPUT);

```

```

pinMode(pinEcho, INPUT);

// Servo init
ESP32PWM::allocateTimer(0);
headServo.setPeriodHertz(50);
headServo.detach();
stopMotors();

// --- Network Init: Home Station Mode ---
WiFi.mode(WIFI_STA);
WiFi.begin("Edge2", "F@rthebest");

Serial.print("Connecting to Wi-Fi: Edge2");
while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
}
Serial.println("\nWi-Fi Connected successfully!");

// Force Wi-Fi transmitter to maximum output for range stability
WiFi.setTxPower(WIFI_POWER_19_5dBm);

Serial.print("Use your phone to navigate to: http://");
Serial.println(WiFi.localIP());

// --- Web Server Setup ---
server.on("/", HTTP_GET, []() {
  Serial.println("Serving root dashboard page...");
  server.send(200, "text/html", htmlDashboard);
});

// Steering Endpoint Routers
server.on("/forward", HTTP_GET, []() {
  currentMode = MODE_MANUAL;
  if (headServo.attached()) headServo.detach();
  moveForward(activeDriveSpeed);
  Serial.println("Manual Drive Command: FORWARD");
  server.send(200, "text/plain", "1");
});
server.on("/backward", HTTP_GET, []() {
  currentMode = MODE_MANUAL;
  if (headServo.attached()) headServo.detach();
  moveBackward(activeDriveSpeed);
  Serial.println("Manual Drive Command: BACKWARD");
  server.send(200, "text/plain", "1");
});

// DYNAMIC MANUAL TURNING CONTROL ROUTER
server.on("/left", HTTP_GET, []() {
  currentMode = MODE_MANUAL;
  if (headServo.attached()) headServo.detach();

  // Apply dynamic speed + 25-point torque boost for skid steering (capped at max 220)
  int manualTurnSpeed = activeDriveSpeed + 25;
  if (manualTurnSpeed > 220) manualTurnSpeed = 220;

  spinLeft(manualTurnSpeed);
  Serial.print("Manual Drive Command: SPIN LEFT (Speed: ");
  Serial.print(manualTurnSpeed);
  Serial.println(")");
  server.send(200, "text/plain", "1");
});

server.on("/right", HTTP_GET, []() {
  currentMode = MODE_MANUAL;
  if (headServo.attached()) headServo.detach();

  // Apply dynamic speed + 25-point torque boost for skid steering (capped at max 220)
  int manualTurnSpeed = activeDriveSpeed + 25;
  if (manualTurnSpeed > 220) manualTurnSpeed = 220;

  spinRight(manualTurnSpeed);
  Serial.print("Manual Drive Command: SPIN RIGHT (Speed: ");
  Serial.print(manualTurnSpeed);
  Serial.println(")");
  server.send(200, "text/plain", "1");
});

server.on("/stop", HTTP_GET, []() {
  currentMode = MODE_MANUAL;
  stopMotors();
  Serial.println("Manual Drive Command: STOP");
  server.send(200, "text/plain", "1");
});

server.on("/set_speed", HTTP_GET, []() {
  if (server.hasArg("val")) {
    activeDriveSpeed = server.arg("val").toInt();
    Serial.print("Manual speed updated to: ");
  }
}

```

```

        Serial.println(activeDriveSpeed);
    }
    server.send(200, "text/plain", "1");
});

server.on("/drift", HTTP_GET, []() {
    currentMode = MODE_MANUAL;
    if (headServo.attached()) headServo.detach();
    server.send(200, "text/plain", "1");
    executeDriftManeuver();
});

// Autopilot Toggles
server.on("/mode_manual", HTTP_GET, []() {
    currentMode = MODE_MANUAL;
    stopMotors();
    if (headServo.attached()) headServo.detach();
    Serial.println("WEB COMMAND: MODE changed to MANUAL steering");
    server.send(200, "text/plain", "Manual Mode");
});
server.on("/mode_obstacle", HTTP_GET, []() {
    currentMode = MODE_OBSTACLE;
    stopMotors();
    Serial.println("WEB COMMAND: MODE changed to OBSTACLE AVOIDANCE");
    server.send(200, "text/plain", "Obstacle Mode");
});
server.on("/mode_line", HTTP_GET, []() {
    currentMode = MODE_LINE;
    stopMotors();
    if (headServo.attached()) headServo.detach();
    Serial.println("WEB COMMAND: MODE changed to LINE FOLLOWING");
    server.send(200, "text/plain", "Line Mode");
});
server.on("/mode_gesture", HTTP_GET, []() {
    currentMode = MODE_GESTURE;
    stopMotors();
    Serial.println("WEB COMMAND: MODE changed to HAND FOLLOWING");
    server.send(200, "text/plain", "Gesture Mode");
});

// Simple 404 handler (Saves memory over 302 redirects)
server.onNotFound([]() {
    server.send(404, "text/plain", "Not Found");
});

server.begin();
Serial.println("Web Server Online.");
}

void loop() {
    server.handleClient();

    switch (currentMode) {
        case MODE_MANUAL:
            break;
        case MODE_OBSTACLE:
            runObstacleAvoidance();
            break;
        case MODE_LINE:
            runLineFollowing();
            break;
        case MODE_GESTURE:
            runHandFollowing();
            break;
    }
}

--- END OF FILE ---

```

Chapter 10: Conclusion and Performance Evaluation

10.1 Conclusion

The project met its central goal: a five-capability robot running entirely on one ESP32-S3. The Minimum-Controller Rule kept the electrical design small and made faults traceable. The most valuable results came from physical debugging: the BMS trip that revealed how capacitor sizing interacts with battery protection, the servo noise that led to dynamic detachment, and the Android routing behaviour that shaped the network design.

10.2 Performance Evaluation

Criterion	Result
Single-controller operation	All modes, web server and safety exits run on the ESP32-S3 with no co-processor
Power stability	No BMS trips with 330 uF / 25 V decoupling
Command responsiveness	yieldDelay() keeps the server serviced; any manual command aborts an autonomous routine at the next check
Servo stability	Random head swings eliminated by dynamic detachment
Control reliability	Phone and robot on one subnet; voice and joystick commands both arrive over local HTTP
Turning capability	Autonomous pivots at 220 PWM overcome floor scrubbing friction

10.3 Observations on the Verified Firmware

The following behaviours of the verified firmware were noted during report preparation. They are recorded for completeness and the firmware has not been altered.

- Manual turns use the slider speed plus 25 (default 120 PWM). Only the autonomous manoeuvres use the fixed 220 PWM TURN_SPEED.
- In the voice parser the forward test is evaluated first and matches any phrase containing "go", so the phrase "go back" is interpreted as forward. The words "back", "backward" and "reverse" work as intended.
- The voice phrases "boost" and "speed" request a /boost route that is not defined in the firmware, so the robot returns its 404 response and takes no action.
- Obstacle detection relies on a 15 ms echo timeout, so distances beyond roughly 2.5 m are reported as no echo.

10.4 Future Work

Within the single-controller architecture, possible refinements include a dedicated /boost route, reordering the voice parser so multi-word phrases are tested first, and wheel encoders on spare GPIOs for closed-loop turn angles.